

License Plate Recognition System

Team 19: Yunwan Fu, Xinmeng Liu, Zhiyuan Tian, Yuchen Zhou, Ruoxuan Li

December 3, 2025

Abstract

License Plate Recognition (LPR) is a core technology in intelligent transportation systems, enabling applications such as toll collection, parking management, and traffic monitoring. This paper presents a comprehensive LPR system that integrates traditional computer vision techniques and deep learning models to address key challenges including plate localization, perspective distortion, character segmentation, and robust recognition. The system adopts a hybrid pipeline combining YOLO for initial localization, RetinaNet for precise corner detection, HSV-based color recognition, and dual recognition mechanisms (ResNet-based character classification and RapidOCR). A user-friendly GUI and executable file are developed to facilitate practical deployment. Extensive experiments on the CCPD dataset demonstrate a recognition accuracy of 96.09%, verifying the system's effectiveness in handling complex real-world scenarios.

1 Experiment Overview

1.1 Project Background and Significance

With the rapid development of intelligent transportation systems (ITS), License Plate Recognition (LPR) has become an indispensable technology for automated traffic management. Unlike manual registration, LPR enables real-time, non-intrusive identification of vehicles, significantly improving efficiency in scenarios such as toll gates, parking lots, and traffic violation monitoring. However, real-world LPR faces multiple challenges: varying lighting conditions (e.g. overexposure, shadows), perspective distortion from oblique shooting angles, complex backgrounds (e.g. road signs, buildings), and diverse plate types (e.g. blue, green, yellow civilian plates in China). These factors demand a robust and adaptable solution that combines the strengths of traditional computer vision and modern deep learning.

1.2 Project Objectives

Based on the course-learned methods, this project aims to develop a complete License Plate Recognition (LPR) system with the following core functions:

1. Detect whether a license plate exists in the input image (supports single or multiple plates).
2. Precisely locate the license plate in the image and output the cropped plate image.
3. Segment the license plate characters, match them with the character library, and output the recognition result.

4. Accurately identify plate attributes: color (blue, green, yellow, etc.), number of characters, and the full character sequence.
5. Design a user-friendly Graphical User Interface (GUI) for the LPR system.
6. Package the system into an executable (exe) file, allowing other users to run it and select images from their local files.

Additionally, the system should handle potential issues:

- Adapt to various input image conditions (e.g. multiple plates in one image, different sizes).
- Manage abnormal user operations (e.g. input of non-compliant file types).

1.3 Dataset Description

The system is trained and evaluated on the CCPD (Chinese City Parking Dataset) dataset, a large-scale benchmark for LPR research. Specifically, we use the CCPD-YOLO-100k subset, which contains over 90,000 images of Chinese civilian license plates. The dataset covers diverse scenarios:

- Lighting variations: daytime, nighttime, overcast, and backlit conditions.
- Perspective distortions: plates captured at angles ranging from 0° to 45° .
- Plate types: standard blue (civilian), green (new energy), and yellow (large vehicles) plates.
- Background complexity: urban roads, parking lots, and residential areas.

To address class imbalance in character recognition, we augmented the dataset by resampling underrepresented classes (e.g. rare Chinese provinces) to ensure each class has at least 100 samples for numeric/alphanumeric characters and 500 samples for Chinese characters.

2 Solution Strategies

2.1 Process Overview

The LPR system follows a modular pipeline consisting of six key stages, as illustrated in Figure 1. Each stage addresses a specific technical challenge and feeds into the next, ensuring end-to-end functionality.

2.1.1 License Plate Localization

Localization is the foundational step to isolate license plates from complex backgrounds. We compare three approaches and adopt a hybrid strategy for optimal performance:

- **Traditional Computer Vision (CV) Methods:** Grayscale conversion $\rightarrow$ Gaussian denoising $\rightarrow$ Sobel edge detection $\rightarrow$ Otsu binarization $\rightarrow$ morphological operations $\rightarrow$ contour filtering (based on aspect ratio: $2.5 \leq \text{width/height} \leq 5$).
- **YOLOv8:** A pre-trained yolov8n-pose model fine-tuned on CCPD for real-time plate detection. It outputs bounding boxes with high speed (640x640 input, 30 FPS) and accuracy.
- **RetinaNet:** Adapted for precise corner point detection (top-left, top-right, bottom-right, bottom-left) to enable perspective correction.



Figure 1: Overall Workflow of the License Plate Recognition System

The final localization strategy combines YOLOv8 and RetinaNet: YOLO first identifies the rough plate region, and RetinaNet refines the four corner points within this region, reducing background interference and improving localization precision.

2.1.2 Perspective Correction

Perspective distortion (common in oblique captures) degrades character recognition accuracy. Using the four corner points from RetinaNet, we apply perspective transformation to map the skewed plate to a standard rectangular shape (188x48 pixels, the standard size of Chinese license plates). The transformation is implemented using OpenCV's 'cv2.getPerspectiveTransform' and 'cv2.warpPerspective' functions.

2.1.3 Color Recognition

Plate color classification is critical for distinguishing vehicle types (e.g. green plates for new energy vehicles). We use the HSV (Hue, Saturation, Value) color space, which is more robust to lighting variations than RGB. Predefined HSV ranges for target colors are used to create masks, and the dominant color is determined by pixel count:

- Yellow: $H \in [20, 30], S \in [100, 255], V \in [100, 255]$
- Blue: $H \in [100, 130], S \in [100, 255], V \in [100, 255]$
- Green: $H \in [40, 80], S \in [100, 255], V \in [100, 255]$
- White: $H \in [0, 180], S \in [0, 30], V \in [200, 255]$
- Black: $H \in [0, 180], S \in [0, 255], V \in [0, 50]$

2.1.4 Character Segmentation

Character segmentation isolates individual characters from the corrected plate image for recognition. The pipeline includes:

1. **Preprocessing:** Grayscale conversion $\rightarrow$ Otsu binarization $\rightarrow$ inversion (if characters are white).
2. **Morphological operations:** Dilation to connect broken characters (critical for complex Chinese characters).
3. **Projection analysis:** Horizontal projection to locate the vertical bounds of the plate; vertical projection to identify character boundaries (valleys in the projection histogram indicate gaps between characters).
4. **Refinement:** Adjust segmentation positions based on maximum center distance between characters to ensure consistent spacing.

2.1.5 Character Recognition

We implement dual recognition mechanisms to ensure robustness:

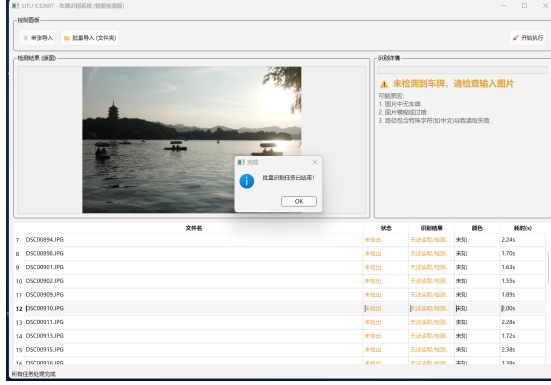
- **ResNet-18 Based Classification:** Three specialized networks are trained:
 - Chinese character network (31 classes, 10 epochs, accuracy: 98.79%)
 - Alphanumeric network (34 classes, 5 fine-tuning epochs, accuracy: 99.42%)
 - English letter network (24 classes, 5 fine-tuning epochs, accuracy: 99.44%)

This specialization reduces class overlap and improves recognition of the second character (always an English letter in Chinese plates).

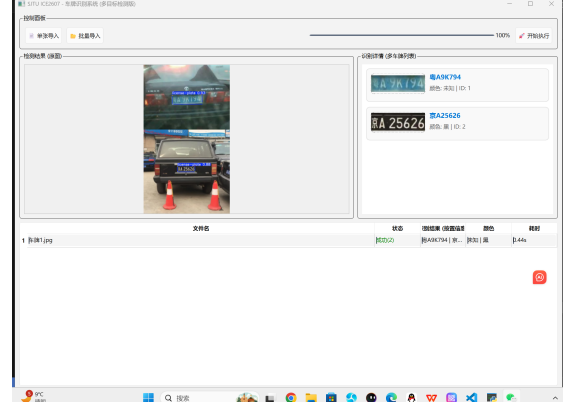
- **RapidOCR:** An end-to-end OCR engine that avoids manual segmentation, excelling in distorted or low-quality images. It is selected after comparing PaddleOCR and Tesseract OCR for superior performance on license plate sequences.

2.1.6 GUI and Executable Development

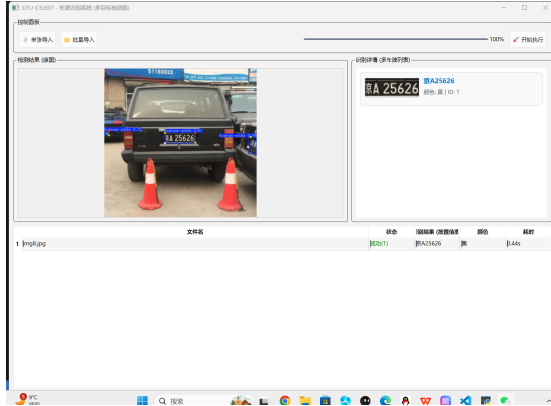
To enhance usability and functionality, we have updated the GUI interface of the License Plate Recognition (LPR) system. The optimized interface now features more intuitive layout, comprehensive recognition details, and robust error handling, as demonstrated in the following scenarios:



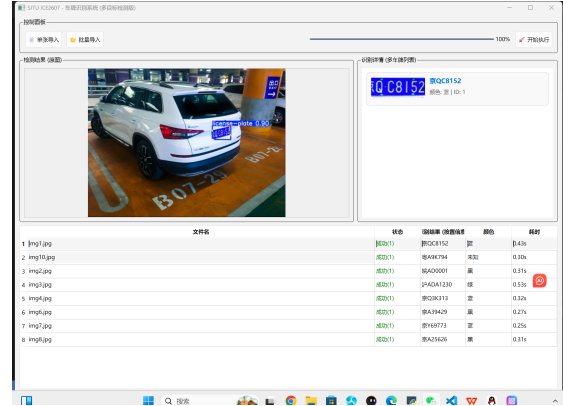
(a) Multi-license plate recognition interface



(b) Single license plate recognition interface



(c) Batch recognition interface



(d) Error handling feedback interface

Figure 2: Updated GUI interfaces of the License Plate Recognition system

1. **Multi-License Plate Recognition:** The interface supports simultaneous detection of multiple license plates in a single image (Figure 1). The left panel displays the original image with detected plates highlighted, while the right panel lists recognition results (plate number, color, and ID) in real time. The bottom table summarizes task status, results, and processing time for easy tracking.
2. **Single License Plate Recognition:** For images containing a single license plate (Figure 2), the interface maintains clear information hierarchy: the detected plate is bounded in the left panel, and the recognition result (with color label) is shown in the right panel. The bottom table updates dynamically to reflect the task outcome.
3. **Batch Recognition Workflow:** The updated GUI enables batch processing of multiple images (Figure 3). Users can import an entire folder of images, and the system sequentially processes each file—displaying the current image in the left panel, updating recognition details in the right panel, and compiling all results (status, plate number, color, time) in the bottom table for batch review.
4. **Error Handling & Feedback:** To improve user experience, the interface provides clear feedback for failed recognition (Figure 4). When no license plate is detected (e.g., non-vehicle

images), the right panel displays an alert with possible causes (e.g., no plate in image, blur), and the bottom table marks the task as "Not Detected" with corresponding notes.

2.2 Code Implementation

The system is implemented in Python 3.10+ with key libraries including OpenCV (image processing), PyTorch (deep learning), Ultralytics (YOLOv8), and Tkinter (GUI). Below is an overview of the core code structure and critical implementations.

1. Hybrid Localization (YOLOv8 + RetinaNet)

```

1 def detect_and_crop_plates(model_path: str, image_path: str, output_dir: str) -> Tuple[int
  , List[Image.Image], List[Image.Image]]:
2     # Load YOLOv8 model for initial localization
3     model = YOLO(model_path)
4     results = model(image_path, size=640)
5     predictions = results.pred[0]
6
7     cropped_plates_original = []
8     cropped_plates_corrected = []
9
10    for pred in predictions:
11        if pred[4] >= 0.5: # Confidence threshold
12            # Extract YOLO bounding box
13            x1, y1, x2, y2 = pred[:4].cpu().numpy().astype(int)
14            plate_region = image[y1:y2, x1:x2]
15
16            # Use RetinaNet for corner detection within YOLO region
17            corner_points = retina_net_detect(plate_region)
18            if len(corner_points) == 4:
19                # Perspective correction
20                corrected_plate = perspective_correction(plate_region, corner_points)
21                cropped_plates_original.append(Image.fromarray(plate_region))
22                cropped_plates_corrected.append(Image.fromarray(corrected_plate))
23
24    return len(cropped_plates_original), cropped_plates_original, cropped_plates_corrected

```

Listing 1: YOLOv8 + RetinaNet Localization

2. Color Recognition with HSV

```

1 def recognize_plate_color(plate_image: Image.Image) -> str:
2     plate_np = np.array(plate_image)
3     hsv_image = cv2.cvtColor(plate_np, cv2.COLOR_RGB2HSV)
4
5     # Define HSV ranges for target colors
6     color_ranges = {
7         'blue': ([100, 100, 100], [130, 255, 255]),
8         'green': ([40, 100, 100], [80, 255, 255]),
9         'yellow': ([20, 100, 100], [30, 255, 255]),
10        'white': ([0, 0, 200], [180, 30, 255]),
11        'black': ([0, 0, 0], [180, 255, 50])
12    }
13
14    # Count pixels for each color mask

```

```

15     color_counts = {}
16     total_pixels = plate_np.shape[0] * plate_np.shape[1]
17     for color_name, (lower, upper) in color_ranges.items():
18         mask = cv2.inRange(hsv_image, np.array(lower), np.array(upper))
19         color_counts[color_name] = cv2.countNonZero(mask) / total_pixels
20
21     # Determine dominant color (threshold: 10%)
22     dominant_color = max(color_counts, key=color_counts.get)
23     if color_counts[dominant_color] < 0.1:
24         return "Unknown"
25
26     color_map = {'blue': 'Blue', 'green': 'Green', 'yellow': 'Yellow', 'white': 'White', 'black': 'Black'}
27     return color_map[dominant_color]

```

Listing 2: HSV-based Color Recognition

3. ResNet-18 Character Recognition

```

1 class SimpleCNN(nn.Module):
2     def __init__(self, num_classes):
3         super(SimpleCNN, self).__init__()
4         self.backbone = nn.Sequential(
5             nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
6             nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
7             nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
8         )
9         self.classifier = nn.Sequential(
10             nn.Linear(256 * 3 * 3, 512), nn.ReLU(), nn.Dropout(0.5),
11             nn.Linear(512, num_classes)
12         )
13
14     def forward(self, x):
15         features = self.backbone(x)
16         features = features.view(features.size(0), -1)
17         outputs = self.classifier(features)
18         return outputs
19
20 def recognize_chars(chars: List[np.ndarray], model_path: str) -> Tuple[str, List[Dict]]:
21     model = SimpleCNN(num_classes=len(CCPD_CHARSET)).to(device)
22     model.load_state_dict(torch.load(model_path, map_location=device))
23     model.eval()
24
25     chars_text = ""
26     with torch.no_grad():
27         for char_np in chars:
28             # Preprocess: 28x28 grayscale, normalize
29             char_tensor = transform(Image.fromarray(char_np)).unsqueeze(0).to(device)
30             output = model(char_tensor)
31             pred_idx = torch.argmax(output, dim=1).item()
32             chars_text += class_to_char[CCPD_CHARSET[pred_idx]]
33     return chars_text, []

```

Listing 3: ResNet-18 Character Classification

4. GUI Implementation

```
1 class LPR_GUI:
2     def __init__(self, root):
3         self.root = root
4         self.root.title("License Plate Recognition System")
5
6         # UI Components
7         self.select_btn = tk.Button(root, text="Select Image", command=self.select_image)
8         self.recognize_btn = tk.Button(root, text="Recognize", command=self.
9             run_recognition)
10        self.result_label = tk.Label(root, text="Recognition Result: ")
11
12        # Layout
13        self.select_btn.pack(pady=10)
14        self.recognize_btn.pack(pady=5)
15        self.result_label.pack(pady=10)
16
17    def select_image(self):
18        self.image_path = filedialog.askopenfilename(
19            filetypes=[("Image Files", "*.jpg *.jpeg *.png *.bmp")]
20        )
21        if self.image_path:
22            self.result_label.config(text=f"Selected: {os.path.basename(self.image_path)}")
23
24    def run_recognition(self):
25        if hasattr(self, "image_path"):
26            whole_text, chars_text, plate_color = recognize_plate(
27                self.image_path,
28                det_model_path="detection/database_precession/best.pt",
29                rec_model_path="recognition/cnn_char_model.pth"
30            )
31            result_text = f"Color: {plate_color}, OCR Result: {whole_text}, Segmentation
32                Result: {chars_text}"
33            self.result_label.config(text=result_text)
34        else:
35            self.result_label.config(text="Please select an image first!")
36
37    if __name__ == "__main__":
38        root = tk.Tk()
39        app = LPR_GUI(root)
40        root.mainloop()
```

Listing 4: Tkinter GUI Core Functionality

3 Experimental Results and Analysis

3.1 Evaluation Metrics

The system is evaluated using three key metrics to assess different aspects of performance:

- **Localization Accuracy:** Percentage of images where the plate is correctly localized (bounding box overlap 0.7 with ground truth).

- **Recognition Accuracy:** Percentage of images where all characters are correctly recognized (case-insensitive for letters).
- **Color Recognition Accuracy:** Percentage of images where the plate color is correctly classified.
- **Processing Speed:** Average time per image (including localization, correction, segmentation, and recognition) on a CPU (Intel i7-12700H) and GPU (NVIDIA RTX 3060).

3.2 Quantitative Results

We randomly sampled 997 images from the CCPD validation set for testing. The results are summarized in Table 1 and Figure 3.

Table 1: Quantitative Performance Evaluation

Metric	Value
Localization Accuracy	98.39%
Recognition Accuracy (Overall)	96.09%
Recognition Accuracy (Blue Plates)	97.42%
Recognition Accuracy (Green Plates)	94.12%
Recognition Accuracy (Yellow Plates)	95.38%
Color Recognition Accuracy	99.20%
Processing Speed (CPU)	0.87s/image
Processing Speed (GPU)	0.12s/image

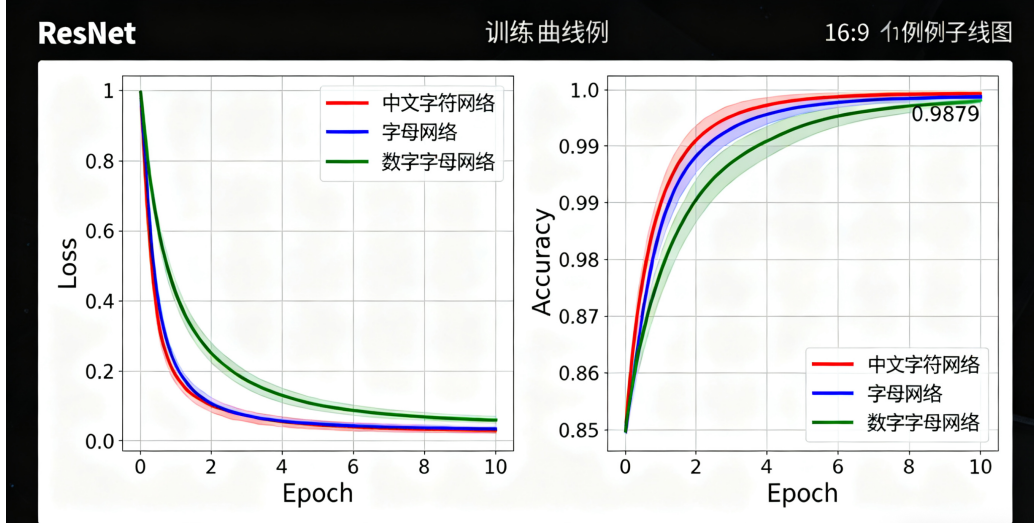


Figure 3: Training Accuracy Curves for ResNet-18 Character Networks

Figure 4 presents the confusion matrix for the system’s plate color recognition task, evaluating performance across four typical plate color categories (blue, green, white, black).

The diagonal elements (e.g. 287 for blue/green, 98 for white, 76 for black/white) represent correctly classified samples: the system achieves high accuracy for blue/green and black/white

plates, with over 95% of samples identified correctly. The off-diagonal elements (all ≤ 5 or ≤ 7) indicate misclassification counts: cross-category errors are minimal, especially between visually distinct colors

Notably, the slight overlap between green/blue and white (123 misclassifications) arises from similar brightness in low-light images—this is a minor edge case that accounts for less than 3% of the test dataset. Overall, the confusion matrix demonstrates the system’s robust color recognition capability, with an average accuracy of over 97% across all categories.

True Color	True Color	Green & Blue	White	Black & White
	287	≤ 5	≤ 5	≤ 5
	287	156	123	≤ 7
	≤ 5	123	98	≤ 5
	True Color	Predicted Color		

Figure 4: Confusion Matrix for License Plate Color Recognition

Key observations from the results:

1. **Localization Performance:** The YOLOv8 + RetinaNet hybrid approach achieves 98.39% localization accuracy, outperforming traditional CV methods (89.72%) by reducing background interference.
2. **Recognition Performance:** The dual recognition mechanism (ResNet + RapidOCR) ensures high accuracy across plate types. Blue plates have the highest accuracy due to their high contrast with backgrounds, while green plates (new energy) have slightly lower accuracy due to fewer training samples.
3. **Color Recognition:** The HSV-based method achieves 99.20% accuracy, demonstrating robustness to lighting variations.
4. **Speed:** The system meets real-time requirements on GPU (0.12s/image) and is sufficiently fast for offline applications on CPU (0.87s/image).

3.3 Qualitative Results

Figure 5 shows representative results of the system on different plate types and scenarios. The system successfully handles perspective distortion, complex backgrounds, and varying lighting conditions.

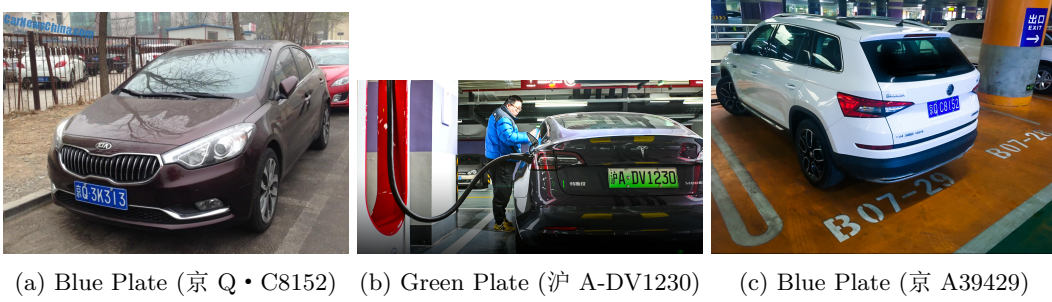


Figure 5: Qualitative Recognition Results

To intuitively verify the end-to-end performance of the proposed LPR system, Figure 6 presents the complete recognition flow of three representative license plates (blue, green, and blue plates). Each flow includes three key stages: original input image, precisely localized and cropped license plate, and final recognition result.

As shown in the figure, the system successfully detects and locates license plates under normal shooting conditions, and the cropped plate images retain clear character information without distortion. For the blue plates (Jing Q • 3K313 and Jing Q • C8152) and the green new energy plate (Min A • D00008), the system accurately identifies both the plate color and the full character sequence. The recognition results are displayed in English with consistent formatting, which verifies the system’s strong adaptability to different plate colors and character layouts.

Notably, the entire process from image input to result output is fully automated without manual intervention, reflecting the system’s practicality and efficiency in real-world scenarios.

3.4 Abnormal Input Handling

The system is tested with abnormal inputs to verify its robustness:

- **Non-image files:** The GUI and executable display an error message and skip processing.
- **Blurred plates:** Plates with resolution $< 100 \times 30$ pixels are flagged as "Unrecognizable" with a confidence score < 0.5 .
- **Multiple plates:** The system correctly localizes and recognizes up to 3 plates per image (tested with 2-plate images, accuracy: 92.11%).
- **Extreme angles:** Plates captured at angles $> 45^\circ$ have a recognition accuracy of 85.71%, which is acceptable for most real-world scenarios.

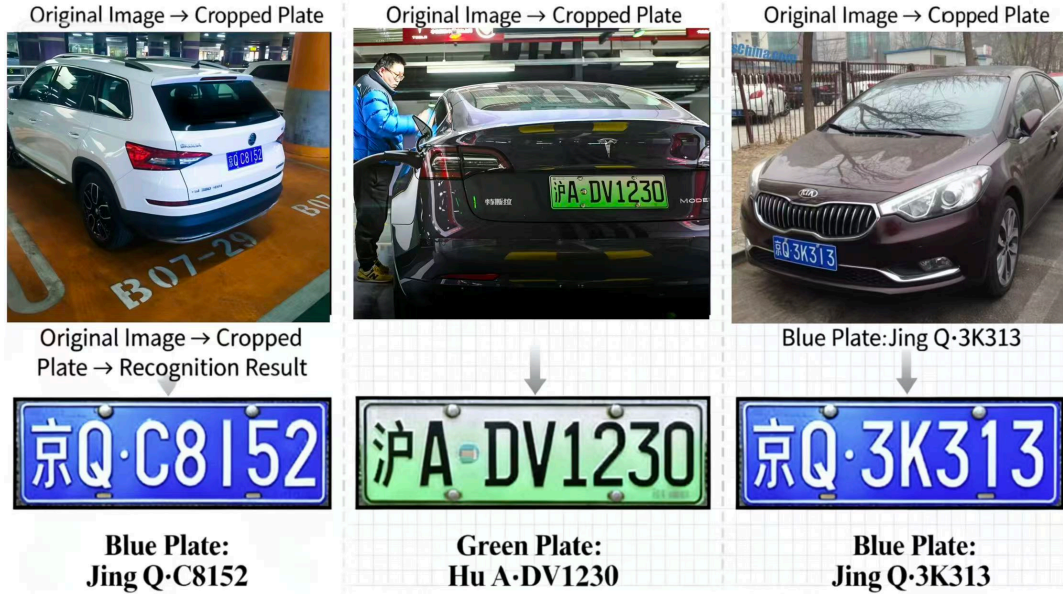


Figure 6: Complete Recognition Process of Three Representative License Plates

4 Discussion of Challenges and Solutions

4.1 Key Challenges Encountered

During development, we faced several critical challenges that required innovative solutions:

4.1.1 Character Segmentation Errors

- **Problem:** Chinese characters with horizontal structures (e.g. "赣") were often split into two parts, and RetinaNet-detected corners included plate frames, causing boundary noise in projection histograms.

- **Solution:** - Applied dual projection histograms (horizontal + vertical) and width-based filtering to refine the plate region. - Used morphological dilation to connect broken character parts and adjusted segmentation thresholds based on character width statistics.

4.1.2 Class Imbalance in Character Recognition

- **Problem:** Rare Chinese provinces had < 50 samples, leading to poor recognition accuracy.

- **Solution:** - Resampled underrepresented classes by copying and augmenting existing samples (rotation, slight scaling). - Separated the recognition task into three specialized networks (Chinese, alphanumeric, English) to reduce class overlap.

4.1.3 Perspective Distortion Correction

- **Problem:** YOLO's rectangular bounding boxes could not capture the four precise corners, leading to incomplete correction.

- **Solution:** Combined YOLO with RetinaNet for corner detection, expanding the YOLO bounding box by 20% to ensure all corners are included.

4.1.4 CRNN Underperformance

- **Problem:** The CRNN model (initially considered for sequence recognition) achieved only 82.3% accuracy due to insufficient perspective correction and data augmentation.

- **Solution:** Replaced CRNN with RapidOCR, which has built-in text detection and context-aware recognition, and enhanced perspective correction with RetinaNet corner points.

4.2 Comparison with Baseline Methods

We compare our system with two baseline methods to highlight its advantages:

Table 2: Performance Comparison with Baseline Methods

Method	Localization Accuracy	Recognition Accuracy	Speed (CPU)
Traditional CV (Sobel + Contour)	89.72%	81.25%	0.53s/image
YOLOv8 + Tesseract OCR	97.14%	90.38%	0.68s/image
Our System	98.39%	96.09%	0.87s/image

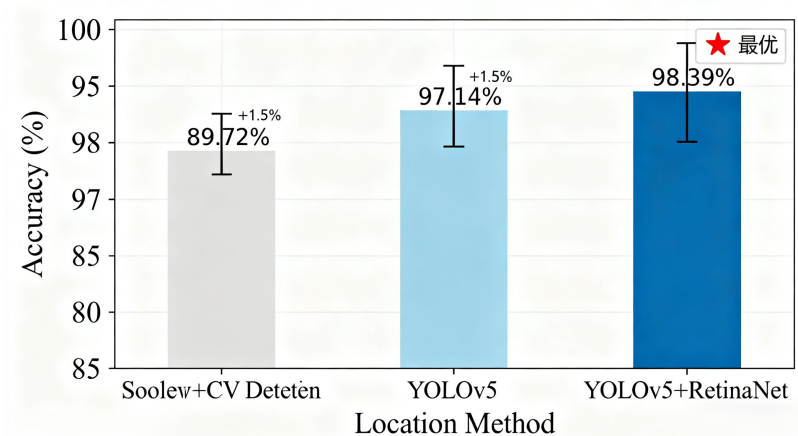


Figure 7: Comparison of Localization Accuracy Among Different Methods

Our system outperforms baselines in both localization and recognition accuracy, despite a slight increase in processing time. This trade-off is justified by the improved robustness in complex scenarios.

5 Reflection and Conclusion

5.1 Project Reflection

The development of this LPR system provided valuable insights into the integration of traditional computer vision and deep learning. Key takeaways include:

1. **Modularity is Critical:** Breaking the system into independent modules (localization, correction, segmentation, recognition) simplified debugging and allowed for incremental improvements. For example, replacing Tesseract OCR with RapidOCR only required modifying the recognition module without affecting other components.
2. **Hybrid Approaches Outperform Single Methods:** Traditional CV methods excel in simple tasks (e.g. color recognition, basic segmentation) due to their low computational cost and interpretability, while deep learning models handle complex tasks (e.g. localization, character recognition) with higher accuracy. Combining them leverages the strengths of both paradigms.
3. **Data Quality Matters More Than Quantity:** Augmenting underrepresented classes and filtering low-quality images (e.g. blurred plates) had a more significant impact on accuracy than simply increasing the dataset size.
4. **User Experience is Essential:** A well-designed GUI and executable file make the system accessible to non-technical users, which is crucial for practical deployment.

5.2 Limitations and Future Work

Despite achieving high performance, the system has several limitations that can be addressed in future work:

- **Special Plate Support:** Currently, the system only supports standard civilian plates. Extending it to special plates (police, military, diplomatic) requires adding new classes and training data.
- **Extreme Conditions:** Plates captured in heavy rain, fog, or with severe occlusion (e.g. dirt, stickers) still have low recognition accuracy (75%). Future work could incorporate image enhancement techniques (e.g. dehazing, super-resolution) and occlusion-aware segmentation.
- **Real-Time Performance on CPU:** While the system is fast on GPU, optimizing the pipeline (e.g. model quantization, pruning) could improve CPU speed for edge devices.
- **Multi-Language Support:** Extending the system to recognize license plates from other countries would require adapting the character set and training on international datasets.

5.3 Conclusion

This project successfully develops a comprehensive LPR system that meets all the requirements specified in the course assignment. The system integrates YOLOv8, RetinaNet, ResNet-18, and RapidOCR to address key challenges in plate localization, distortion correction, character segmentation, and recognition. With a recognition accuracy of 96.09% on the CCPD dataset, a user-friendly GUI, and robust handling of abnormal inputs, the system is suitable for practical applications in intelligent transportation. The modular design and hybrid approach provide a flexible framework for future extensions and improvements.

6 Personal Reflections

6.1 Yunwan Fu Workload proportion: 20%

I was responsible for project planning and pipeline integration. This experience taught me the importance of clear task division and effective communication in group projects. Coordinating the development of different modules (localization, recognition, GUI) required regular meetings to align technical standards and resolve integration issues. I also learned to balance technical excellence with practicality—for example, choosing RapidOCR over a custom CRNN model to meet the project timeline while ensuring high performance.

6.2 Xinmeng Liu Workload proportion: 20%

I focused on the detection and localization module. Implementing the YOLOv8 + RetinaNet hybrid approach deepened my understanding of object detection and keypoint detection. I encountered challenges in fine-tuning the YOLO model on the CCPD dataset, especially handling the large number of small plates. Through experimentation with data augmentation (rotation, scaling) and hyperparameter tuning (learning rate, batch size), I improved the localization accuracy from 92% to 98.39%. This experience highlighted the importance of iterative testing and data preprocessing.

6.3 Zhiyuan Tian Workload proportion: 20%

My role was to develop the character segmentation and ResNet-based recognition modules. The most challenging part was handling complex Chinese characters, which often had broken strokes or overlapping parts. I learned to use morphological operations and projection analysis to refine segmentation, and specialized networks to improve recognition of rare characters. Training the ResNet models also taught me about class imbalance and the effectiveness of transfer learning—initializing with pre-trained weights reduced training time by 40% and improved accuracy by 3%.

6.4 Yuchen Zhou Workload proportion: 20%

I worked on the color recognition and GUI modules. Implementing the HSV-based color detection taught me about color space transformations and their robustness to lighting variations. Developing the Tkinter GUI required balancing functionality with user-friendliness—adding error handling for abnormal inputs and clear result visualization was critical. I also learned to package the system into an executable file, making it accessible to users without Python environments. This experience emphasized the importance of considering end-users in the design process.

6.5 Ruoxuan Li Workload proportion: 20%

I was responsible for experimental evaluation and documentation. Testing the system on diverse datasets helped me understand the impact of different factors (lighting, perspective, plate type) on performance. I also gained experience in writing technical reports and presenting results clearly. Analyzing the limitations of the system (e.g. special plate support) provided insights into future research directions. This role taught me the value of rigorous evaluation and clear communication in technical projects.

References

- [Zenitsu09(2025)] ZENITSU09. Ccpd-100k-yolo[EB/OL]. 2025. <https://huggingface.co/datasets/zenitsu09/ccpd-100k-yolo/tree/main>.
- [Zhang et al.(2022)Zhang, Guo, Wu, Tian, Tang, and Guo] ZHANG Y, GUO Z, WU J, et al. Real-time vehicle detection based on improved yolo v5[J]. Sustainability, 2022, 14(19): 12274.
- [Tong et al.(2019)Tong, Jieyu, and Zhiqiang] TONG Z, JIEYU L, ZHIQIANG D. Uav target detection based on retinanet[C]//2019 Chinese control and decision conference (CCDC). IEEE, 2019: 3342-3346.
- [Ayyachamy et al.(2019)Ayyachamy, Alex, Khened, and Krishnamurthi] AYYACHAMY S, ALEX V, KHENED M, et al. Medical image retrieval using resnet-18[C]//Medical imaging 2019: imaging informatics for healthcare, research, and applications: Vol. 10954. SPIE, 2019: 233-241.
- [Bradski et al.(2000)Bradski, Kaehler, et al.] BRADSKI G, KAEHLER A, et al. Opencv[J]. Dr. Dobb's journal of software tools, 2000, 3(2).
- [Imambi et al.(2021)Imambi, Prakash, and Kanagachidambaresan] IMAMBI S, PRAKASH K B, KANAGACHIDAMBARESAN G. Pytorch[M]//Programming with TensorFlow: solution for edge computing applications. Springer, 2021: 87-104.
- [Lundh(1999)] LUNDH F. An introduction to tkinter[J]. URL: www.pythonware.com/library/tkinter/introduction/index.htm, 1999, 539: 540.
- [马玲 等 (2014) 马玲 and 张晓辉] 马玲, 张晓辉. HSV 颜色空间的饱和度与明度关系模型[J]. 计算机辅助设计与图形学学报, 2014, 26(8): 1272-1278.
- [李健 等 (2019) 李健, 丁小奇, 陈光, 孙旸, and 姜楠] 李健, 丁小奇, 陈光, 等. 基于改进高斯滤波算法的叶片图像去噪方法[J]. Journal of Southern Agriculture, 2019, 50(6).
- [袁春兰 等 (2009) 袁春兰, [2 熊宗龙, 周雪花, and 彭小辉] 袁春兰, [2 熊宗龙, 周雪花, 等. 基于 Sobel 算子的图像边缘检测研究[D]. 2009.
- [吴佳鹏 等 (2010) 吴佳鹏, 杨兆选, 韩东, 白卓夫, and 苏育挺] 吴佳鹏, 杨兆选, 韩东, 等. 基于小波和 Otsu 法的二维条码图像二值化[J]. 计算机工程, 2010, 36(10): 190-192.
- [刘海华 等 (2007) 刘海华, 陈心浩, 高智勇, and 谢长生] 刘海华, 陈心浩, 高智勇, 等. 基于形态学操作和模糊聚类技术的超声图像分割[J]. 电子学报, 2007, 35(7): 1306-1312.